Part 01 : Theoretical Questions

Q1 : What is abstraction in OOP? How is it different from encapsulation? Give a real-world example (not from the session) that shows the difference between the two.

Q2 : What is the difference between an abstract class and an interface? Give at least four differences. When would you choose one over the other?

Q3 : Look at the following code and answer the questions below:

```
public abstract class Appliance
{
    public string Brand { get; set; }

    protected Appliance(string brand) { Brand = brand; }

    public abstract double PowerConsumption();

    public virtual string Status() => "Standby";

    public string Label() => $"{Brand} - {PowerConsumption()}W";
}

public class WashingMachine : Appliance
{
    public WashingMachine(string brand) : base(brand) { }
    public override double PowerConsumption() => 500;
    public override string Status() => "Washing";
}
```

```
public class Toaster : Appliance
{
    public Toaster(string brand) : base(brand) { }
    public override double PowerConsumption() => 800;
}
```

a) Can you write: Appliance a = new Appliance("LG"); ? Why or why not?

b) What is the difference between the three methods: PowerConsumption(), Status(), and Label()? Why did the designer make each one abstract, virtual, or concrete?

c) If you call Status() on a Toaster object, what will it return? Why?

Q4 : Look at the following code and answer the questions below:

```
// File: Calculator.cs
public partial class Calculator
{
    public double LastResult { get; private set; }
    partial void OnCalculated(double result);

    public double Add(double a, double b)
    {
        LastResult = a + b;
        OnCalculated(LastResult);
        return LastResult;
    }
}

// File: Calculator.Logging.cs
public partial class Calculator
{
    partial void OnCalculated(double result)
    {
        Console.WriteLine($"Log: result = {result}");
    }
}

// File: DoubleExtensions.cs
public static class DoubleExtensions
{
    public static string ToCurrency(this double value)
        => $"${value:F2}";
}
```

a) What is a partial class? Why would a developer split Calculator into two files?

b) What is a partial method? What happens if the OnCalculated() implementation in Calculator.Logging.cs is deleted — will the code still compile? Why?

c) What is an extension method? What are the three rules for writing one?

d) What will the following code print?

```
Calculator calc = new Calculator();
double result = calc.Add(19.5, 0.5);
Console.WriteLine(result.ToCurrency());
```

ROUTE

Part 02 : Practical (Extending the Movie Ticket Booking System)

In the previous assignments, you built a Movie Ticket Booking System with inheritance, polymorphism, interfaces, and object copying. Now you will apply abstraction, abstract classes, partial classes, and extension methods to improve the design.

User Story :

The cinema manager has reviewed the system and requested three improvements:

1. No Plain Tickets — The manager noticed that in theory, someone could create a plain Ticket object that doesn't belong to any category. This should never happen — every ticket must be either Standard, VIP, or IMAX. The system should enforce this at the design level so the compiler itself prevents creating a plain Ticket. At the same time, there are some calculations that every ticket type must provide its own version of (like how the final price is calculated), while other behaviors (like booking and cancellation) should stay shared across all types.

2. Organized Cinema Code — The Cinema class is growing too large with ticket management, reporting, and projector control all in one file. The development team wants to split it into multiple files for better organization, without creating separate classes. One file should handle ticket operations (adding tickets, booking), and another should handle reporting (printing all tickets, showing statistics). Both files should contribute to a single Cinema class.

3. Useful Utilities Without Modifying Existing Classes — The team needs to add some handy features to the existing Ticket types without touching their source code. For example: a method to generate a formatted receipt string from any ticket, and a method that takes an array of tickets and returns the total revenue. These should feel like they belong to the Ticket class when you call them, even though they are defined elsewhere.

ROUTE

Requirements :

Your solution must demonstrate the following concepts from this session:

- Making the base Ticket class abstract — with at least one abstract method that each child class must implement
- Using abstract, virtual, and concrete members together in the abstract class
- Using the abstract class for polymorphism (e.g. an array of Ticket holding different types)
- Splitting the Cinema class using partial classes (at least two files)
- Creating a static class with at least two extension methods for Ticket or Ticket-related types
- Calling the extension methods naturally on objects (not as static method calls)

In Main, demonstrate :

a. Try to create a plain Ticket object and show (in a comment) that the compiler prevents it.

b. Create one of each ticket type with hardcoded data. Book all three.

c. Add all three tickets to a Cinema and print them all (the print should go through the Cinema's reporting partial file).

d. Use polymorphism: loop through a Ticket[] array and call the abstract method on each to show each type calculates differently.

e. Call an extension method on a ticket to generate a receipt string and print it.

f. Call an extension method on the ticket array to calculate and print the total revenue.

g. Close the Cinema.

ROUTE

Expected Output (Example) :

```
=== Cinema Opened ===
 Projector ON

 // Ticket t = new Ticket("Test", 100);  // ERROR: Cannot create instance of abstract
type 'Ticket'

 --- All Tickets (from Cinema.Reporting) ---
 [Ticket #1] Inception | Standard | Seat: A5 | Price: 80 | Final: 91.20 | Booked: Yes
 [Ticket #2] Avengers | VIP | Lounge: Yes | Fee: 50 | Price: 200 | Final: 285.00 |
Booked: Yes
 [Ticket #3] Dune | IMAX | 3D: Yes | Price: 130 | Final: 148.20 | Booked: Yes

 --- Polymorphism: Final Price per Ticket ---
 StandardTicket => Final Price: 91.20
 VIPTicket => Final Price: 285.00
 IMAXTicket => Final Price: 148.20

 --- Extension Method: Receipt ---
 ========== RECEIPT ==========
   Movie    : Avengers
   Type     : VIPTicket
   Price    : 200
   Final    : 285.00
   Status   : Booked
 ============================

 --- Extension Method: Total Revenue ---
 Total Revenue: 524.40

 Projector OFF
 === Cinema Closed ===
```